

Pertemuan 5

Kompleksitas

Algoritma dan Big-O

Pemrograman untuk Ilmu Data dan Komputasi

MA25-21008 Kelas RA dan RB

Rifky Fauzi

Triyana Muliawati, S.Si., M.Si.

Program Studi Matematika
Institut Teknologi Sumatera
Semester Ganjil 2026/2027

Alur pertemuan

- Dari terminasi ke jumlah langkah.
- Menurunkan $T(n)$ dari kode.
- Mengambil orde dominan: Big-O.
- Membandingkan beberapa algoritma anagram.
- Membaca biaya operasi list melalui eksperimen.

Dari M4: program berhenti, lalu berapa kerjanya?

Pertanyaan M4

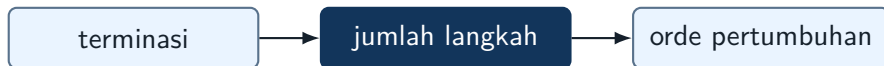
Program ini berhenti atau tidak?

```
i = n
while i > 0:
    i = i // 2
```

Pertanyaan M5

Jika berhenti, berapa banyak langkah yang kira-kira dilakukan saat n membesar?

```
test = 0
for i in range(n):
    for j in range(n):
        test = test + i*j
```



Kasus pembuka: menjumlahkan $1 + 2 + \dots + n$

Algoritma A: akumulasi

```
def jumlah_loop(n):  
    s = 0  
    for i in range(1, n+1):  
        s = s + i  
    return s
```

Algoritma B: rumus

```
def jumlah_rumus(n):  
    return n*(n+1)//2
```

Diskusi di papan

Keduanya memberi jawaban sama. Untuk $n = 10$, berapa kali baris `s = s + i` dijalankan? Bagaimana untuk $n = 1000$?

Kasus pembuka: menjumlahkan $1 + 2 + \dots + n$

Algoritma A: akumulasi

```
def jumlah_loop(n):  
    s = 0  
    for i in range(1, n+1):  
        s = s + i  
    return s
```

Algoritma B: rumus

```
def jumlah_rumus(n):  
    return n*(n+1)//2
```

Diskusi di papan

Keduanya memberi jawaban sama. Untuk $n = 10$, berapa kali baris $s = s + i$ dijalankan? Bagaimana untuk $n = 1000$?

Pada algoritma A, kerja utama bertambah mengikuti banyak suku. Pada algoritma B, tidak ada loop terhadap n .

Dari jumlah operasi ke Big-O

Langkah analisis

- 1 Tentukan ukuran input: biasanya n .
- 2 Pilih operasi yang dihitung.
- 3 Turunkan $T(n)$.
- 4 Ambil suku yang dominan saat n besar.

Contoh

$$T(n) = 5n^2 + 27n + 1005$$

Langkah analisis

- 1 Tentukan ukuran input: biasanya n .
- 2 Pilih operasi yang dihitung.
- 3 Turunkan $T(n)$.
- 4 Ambil suku yang dominan saat n besar.

Contoh

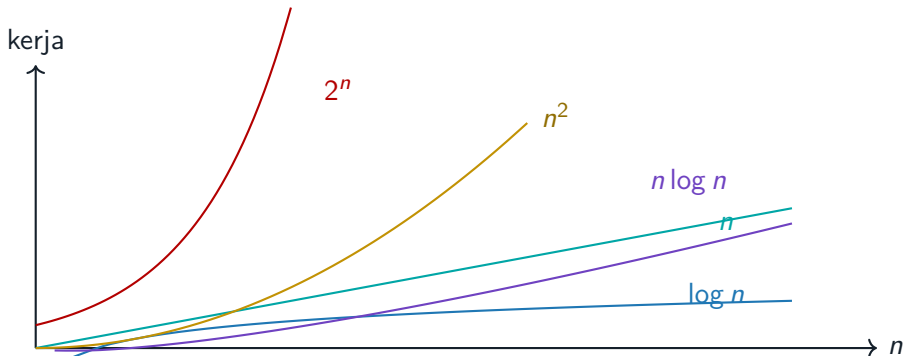
$$T(n) = 5n^2 + 27n + 1005$$

$$T(n) = O(n^2)$$

Yang diabaikan

Konstanta dan suku yang tumbuh lebih lambat. Bukan karena tidak ada, tetapi karena pengaruhnya kalah saat n besar.

Tangga pertumbuhan yang sering muncul



Catatan mengajar

Untuk n kecil, grafik belum terlalu berpisah. Saat n membesar, orde pertumbuhan mulai menentukan pilihan algoritma.

Contoh papan 1: nested loop

```
test = 0
for i in range(n):
    for j in range(n):
        test = test + i*j
```

Hitung manual

- Loop luar: n kali.
- Untuk setiap i , loop dalam: n kali.
- Baris terdalam: $n \times n$ kali.

Contoh papan 1: nested loop

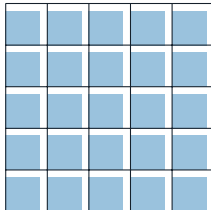
```
test = 0
for i in range(n):
    for j in range(n):
        test = test + i*j
```

Hitung manual

- Loop luar: n kali.
- Untuk setiap i , loop dalam: n kali.
- Baris terdalam: $n \times n$ kali.

Kesimpulan

$$T(n) \approx n^2 \Rightarrow O(n^2).$$



$n \times n$ kotak kerja

Contoh papan 2: dua loop berurutan

```
test = 0
for i in range(n):
    test = test + 1
for j in range(n):
    test = test - 1
```

Jangan langsung melihat dua loop sebagai kuadrat

Kedua loop tidak bersarang. Loop kedua mulai setelah loop pertama selesai.

Contoh papan 2: dua loop berurutan

```
test = 0
for i in range(n):
    test = test + 1
for j in range(n):
    test = test - 1
```

Jangan langsung melihat dua loop sebagai kuadrat

Kedua loop tidak bersarang. Loop kedua mulai setelah loop pertama selesai.

Hitungan

$$T(n) = n + n = 2n.$$

Konstanta 2 tidak mengubah orde.

$$T(n) = O(n).$$

Contoh papan 3: setiap langkah dibagi dua

```
i = n
while i > 0:
    k = 2 + 2
    i = i // 2
```

Ambil contoh $n = 32$

$32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1 \rightarrow 0$

Contoh papan 3: setiap langkah dibagi dua

```
i = n
while i > 0:
    k = 2 + 2
    i = i // 2
```

Ambil contoh $n = 32$

$32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1 \rightarrow 0$

Banyak pembagian dua sampai mencapai 1 berkaitan dengan $\log_2 n$.

$$T(n) = O(\log n).$$



Contoh papan 4: jumlah segitiga

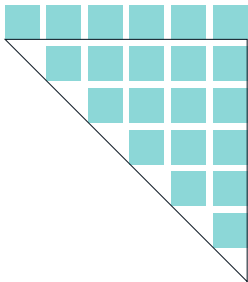
```
s = 0
for i in range(n):
    for j in range(i + 1):
        s += 1
```

Tuliskan banyak eksekusi

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$

Contoh papan 4: jumlah segitiga

```
s = 0
for i in range(n):
    for j in range(i + 1):
        s += 1
```



Tuliskan banyak eksekusi

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$

$$T(n) = \frac{1}{2}n^2 + \frac{1}{2}n = O(n^2).$$

bukan persegi penuh, tetapi tetap orde n^2

Loop versus rumus tertutup

Loop

$$S(n) = 1 + 2 + \dots + n$$

$$T(n) = n \Rightarrow O(n)$$

Rumus

$$S(n) = \frac{n(n+1)}{2}$$

$$T(n) = 1 \Rightarrow O(1)$$

Poin penting

Benchmark waktu boleh dilakukan, tetapi Big-O menilai pertumbuhan kerja algoritma terhadap ukuran input. Waktu aktual dipengaruhi mesin, sistem operasi, dan kondisi saat percobaan.

Kasus utama: apakah dua daftar karakter anagram?

earth

heart

jawaban: sama susunan huruf

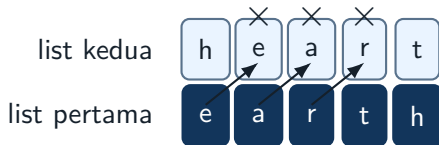
Pertanyaan yang dikerjakan

Diberikan dua list/string dengan panjang sama. Kita ingin fungsi bernilai True jika keduanya berisi elemen yang sama dengan frekuensi yang sama.

Tiga strategi

- 1 Periksa satu per satu dan coret yang sudah dipakai.
- 2 Urutkan, lalu bandingkan.
- 3 Hitung frekuensi setiap karakter.

Anagram A: periksa lalu coret

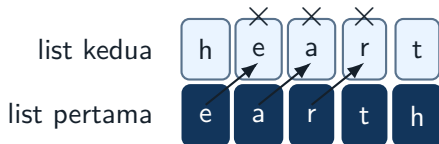


Analisis

Karakter pertama mungkin mencari hampir sepanjang list kedua. Begitu seterusnya.

$$1 + 2 + \dots + n = \frac{n(n+1)}{2}$$

Anagram A: periksa lalu coret



Analisis

Karakter pertama mungkin mencari hampir sepanjang list kedua. Begitu seterusnya.

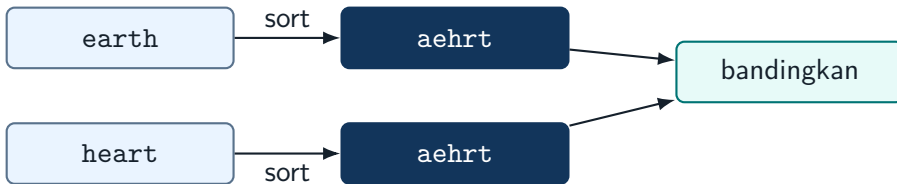
$$1 + 2 + \dots + n = \frac{n(n+1)}{2}$$

$$O(n^2)$$

Catatan

Perlu tempat yang dapat diubah, karena string bersifat immutable. Maka string sering diubah dulu menjadi list.

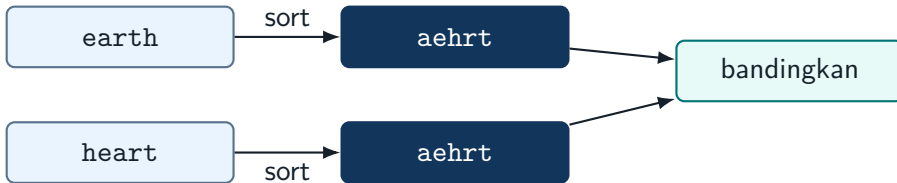
Anagram B: urutkan lalu bandingkan



Bagian kerja

- Mengurutkan list pertama.
- Mengurutkan list kedua.
- Membandingkan posisi demi posisi.

Anagram B: urutkan lalu bandingkan



Bagian kerja

- Mengurutkan list pertama.
- Mengurutkan list kedua.
- Membandingkan posisi demi posisi.

Analisis

Perbandingan setelah sort adalah $O(n)$.
Biaya dominan berasal dari sorting, biasanya $O(n \log n)$.

Anagram C: hitung frekuensi

huruf	a	b	c	d	e	...	h	r	t
count	1	0	0	0	1	...	1	1	1

Ide

Hitung berapa kali setiap karakter muncul pada kedua daftar. Jika semua hitungan sama, maka anagram.

Anagram C: hitung frekuensi

huruf	a	b	c	d	e	...	h	r	t
count	1	0	0	0	1	...	1	1	1

Ide

Hitung berapa kali setiap karakter muncul pada kedua daftar. Jika semua hitungan sama, maka anagram.

Analisis

Dua kali membaca input dan satu kali membandingkan counter:

$$T(n) = 2n + 26 = O(n).$$

Ruang tambahan dipakai untuk counter.

Tiga algoritma, satu masalah

Strategi	Gagasan	Waktu	Ruang tambahan
Coret satu per satu	cari pasangan karakter dalam list kedua	$O(n^2)$	list penanda
Sort lalu bandingkan	urutkan dua list, lalu cocokkan posisi	$O(n \log n)$	tergantung sorting
Hitung frekuensi	simpan jumlah kemunculan setiap karakter	$O(n)$	counter

Pertanyaan kelas

Jika alfabet hanya 26 huruf, strategi counter tampak murah. Bagaimana jika simbol yang mungkin jutaan jenis? Bagian mana yang berubah?

Operasi list: beberapa murah, beberapa mahal

Operasi	Orde	Catatan
<code>x[i]</code>	$O(1)$	akses langsung ke indeks
<code>x.append(a)</code>	$O(1)$	menambah di belakang
<code>x.pop()</code>	$O(1)$	mengambil elemen terakhir
<code>x.pop(0)</code>	$O(n)$	elemen lain harus bergeser
<code>a in x</code>	$O(n)$	pencarian berurutan
<code>x.sort()</code>	$O(n \log n)$	biaya sorting dominan

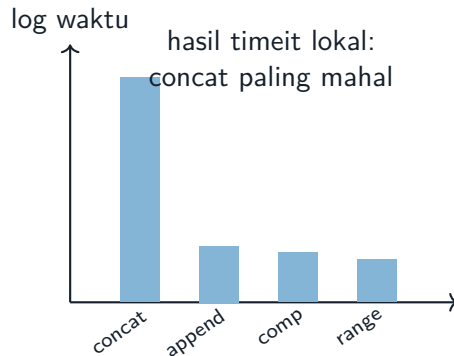
Kaitkan dengan M3

List mudah dipakai untuk data berurutan, tetapi tidak semua operasi pada list cocok untuk data besar.

Eksperimen Python: membuat list

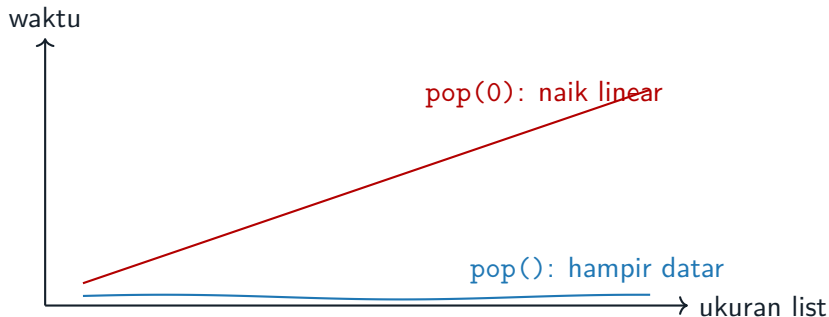
Empat cara

- Konkatenasi: `l = l + [i]`
- `append`
- List comprehension
- `list(range(n))`



cara	concat	append	comp	range
waktu 200 kali, $n = 1000$	0.168s	0.0046s	0.0037s	0.0037s

Eksperimen Python: `pop()` dan `pop(0)`



Penjelasan di papan

`pop()` mengambil elemen belakang. `pop(0)` mengambil elemen depan sehingga elemen lain harus digeser satu posisi.

Tentukan Big-O

```
test = 0
for i in range(n):
    for j in range(n):
        test = test + i*j
```

Self Check 1

Tentukan Big-O

```
test = 0
for i in range(n):
    for j in range(n):
        test = test + i*j
```

Hint

Gambar kotak kerja $n \times n$. Baris terdalam dijalankan satu kali untuk setiap pasangan (i, j) .

Self Check 2

A

```
for i in range(n):  
    test = test + 1  
for j in range(n):  
    test = test - 1
```

Self Check 2

A

```
for i in range(n):  
    test = test + 1  
for j in range(n):  
    test = test - 1
```

B

```
i = n  
while i > 0:  
    k = 2 + 2  
    i = i // 2
```

Hint A

Dua loop ini berurutan, bukan bersarang.

Self Check 2

A

```
for i in range(n):  
    test = test + 1  
for j in range(n):  
    test = test - 1
```

B

```
i = n  
while i > 0:  
    k = 2 + 2  
    i = i // 2
```

Hint A

Dua loop ini berurutan, bukan bersarang.

Hint B

Tuliskan contoh 32, 16, 8, 4, 2, 1. Banyak langkah terkait pembagian dua berulang.

Self Check 3

Pilih operasi list yang bukan $O(1)$

- ① `list.pop(0)`
- ② `list.pop()`
- ③ `list.append(x)`
- ④ `list[10]`

Self Check 3

Pilih operasi list yang bukan $O(1)$

- ❶ `list.pop(0)`
- ❷ `list.pop()`
- ❸ `list.append(x)`
- ❹ `list[10]`

Hint

Pikirkan apakah operasi tersebut perlu menggeser banyak elemen atau cukup mengakses posisi tertentu.

Pertanyaan lanjutan

Mengapa `x in list` dan `x in dict` dapat memiliki perilaku waktu yang berbeda?

Penutup dan latihan M5

Yang perlu dikuasai

- Menurunkan $T(n)$ dari potongan kode.
- Membedakan loop bersarang dan loop berurutan.
- Mengenali $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, dan $O(n^2)$.
- Menganalisis beberapa algoritma untuk masalah yang sama.
- Menafsirkan eksperimen waktu sebagai bukti pendukung, bukan definisi Big-O.

Latihan yang disiapkan

Kerjakan LKM M5.1–M5.10. Untuk setiap soal, tulis ukuran input, operasi yang dihitung, $T(n)$ atau argumen pertumbuhan, lalu simpulkan Big-O.

- Bradley N. Miller dan David L. Ranum. *Problem Solving with Algorithms and Data Structures Using Python*. Release 3.0. Bab 2: Algorithm Analysis.
- Lembar Kerja Mandiri Pemrograman untuk Ilmu Data dan Komputasi, Bab M5: Kompleksitas Algoritma Big-O.